

Programming Assignment

Reverse Engineering a Java Program into Python

From Java to Python

In this assignment, you will take an existing Java application and reverse engineer it into Python. Your goal is not simply to “translate line by line,” but to study how the Java program is designed and then rebuild that same design in Python using correct Python syntax, conventions, and object-oriented practices. This assignment is meant to help you strengthen your understanding of Python while also deepening your grasp of class design, inheritance, constructors, method behavior, encapsulation, and string representation. As you work, pay close attention to how the original Java program uses a superclass, subclasses, state, and behavior. Then think carefully about how those same ideas should be expressed in Python, which often uses less syntax and a different style even when the underlying object-oriented concepts remain the same.

You will begin with the provided Java Library project. Study the code carefully and identify the purpose of each class, each attribute, each constructor, and each method. Your Python version must preserve the overall logic of the original application. In particular, your Python program should include a `LibraryItem` base class and the related subclasses shown in the Java program. The Python version should support the same general behaviors, including checking out an item, returning an item, and representing an item as a string. Since the Java version uses a `toString()` method, your Python version must implement `__str__()` so that printing an object displays meaningful information. Your Python program should also include a main driver section that creates objects and demonstrates that the program works correctly. As in the Java version, your classes should not be responsible for speaking to the user. In other words, do not place `print()` statements or `input()` calls inside your domain classes. Any interaction or output should occur in the main portion of the program.

As you rewrite the application, focus on writing Python in a Pythonic way rather than forcing Java syntax into Python. That means using Python class definitions, the `__init__()` constructor, the `self` parameter, and the `__str__()` method properly. It also means thinking carefully about encapsulation. For this assignment, use private instance attributes in a Pythonic way by using double-underscore name mangling such as `self.__title` rather than protected-style attributes such as `self._title`. Your methods should return values where appropriate, just as the Java methods return boolean results for actions like `checkout` and `return`. You should also think about how inheritance works in Python and how subclass constructors call the parent constructor. The purpose of the assignment is to help you see that while Java and Python look different, many object-oriented design ideas transfer directly across languages.

When you submit your work, include your Python source files (GitHub link) and a short reflection. In your reflection, explain what parts of the Java program were easiest to translate into Python, what parts required the most thought, and what you learned about the differences between Java and Python. You should comment on topics such as constructors, private attributes, inheritance, and the difference between Java’s `toString()` and Python’s `__str__()`. Strong submissions will show that you understood the original design, recreated it accurately in Python, and made thoughtful choices about how to express the program using proper Python conventions rather than simply copying Java structure mechanically.

Programming Assignment

Reverse Engineering a Java Program into Python

Your work will be evaluated based on correctness, completeness, object-oriented design, and use of Python conventions. A strong submission will preserve the original program's class relationships and behavior, use private attributes appropriately, implement `__str__()` correctly, keep output in the main portion of the program, and demonstrate clear understanding of how Python expresses object-oriented ideas differently from Java. Your reflection should show real engagement with the reverse-engineering process and should make clear that you are learning not just a new syntax, but a new way of thinking about program design across languages.